

100 Days AI & DSA Engineer Program

Your Plain-English Guide to Every Module

What it is. Why it matters. How everything connects.



This document is written for you — whether you are a student, a parent, a career-switcher, or simply someone curious about what this program teaches. No tech background needed. We explain everything in plain language, and we show you how every piece fits into one big picture.

Before You Begin: How to Read This Document

Think of becoming an AI Engineer the same way you would think about building a house.

You cannot hang a door before you build the walls. You cannot build walls before you lay the foundation. And you cannot lay a foundation without first understanding the land you are building on.

This program follows exactly that logic. Each phase teaches you something new — but it always connects to what came before and prepares you for what comes next. This guide will show you those connections clearly, so you always understand not just what you are learning, but why you are learning it right now.



Foundation = Mathematics & Logic thinking | Walls = Machine Learning & Deep Learning | Roof = Transformers & GenAI | Interior = Real Projects & Deployment

Throughout this document, you will see:

- WHY sections — explaining the purpose of each topic in everyday language
- WHAT sections — explaining what the topic actually teaches
- HOW IT CONNECTS — showing exactly how this topic leads to the next one



The Big Picture

What this program is, and why it is designed the way it is

What Is This Program?

The 100 Days AI & DSA Engineer Program is a structured, daily learning experience. Every single day for 100 days, you spend:

- 1 hour on DSA — Data Structures & Algorithms (problem-solving and coding skills)
- 1 hour on AI — Artificial Intelligence (understanding and building intelligent systems)

At the end of 100 days, you will have logged 100 hours of problem-solving practice and 100 hours of AI engineering knowledge. That is the equivalent of many months of self-study, packed into a focused, guided sprint.

Why Two Subjects at the Same Time?

Most people think: 'I want to work in AI, so I should only study AI.' But the top companies that hire AI engineers — Google, Amazon, Meta, Anthropic, and others — require you to pass coding interviews as well.

Coding interviews test your ability to solve logic and math puzzles using code. This is called DSA — Data Structures & Algorithms. These companies do not hire you just because you know AI. They hire you because you can think, code, and solve problems.



Think of it this way: DSA is your driver's license test. AI knowledge is your actual driving skill. You need both to get the job.

The Core Philosophy: First-Principles Teaching

Most online courses teach you how to press buttons. They show you how to use a tool without explaining why it exists. This program does the opposite.

Every topic starts with one question: WHY does this exist? Only then do we move to HOW it works, and finally to HOW TO USE it.

This matters because the AI field changes rapidly. New tools appear every few months. If you only learn tools, you become outdated quickly. If you understand the principles behind the tools, you can adapt to anything new.



DSA — Data Structures & Algorithms

The problem-solving engine that runs every single day

What Is DSA and Why Should You Care?

Imagine you are asked to find a specific book in a library. One approach is to walk through every aisle randomly until you find it. Another approach is to go directly to the catalog, find the section number, and walk straight to it. The second approach is faster — it uses a smarter strategy.

This is essentially what DSA teaches. It is the science of solving problems efficiently — choosing the right approach so your code runs fast and does not waste memory.



DSA is not about memorizing syntax. It is about training your brain to look at any problem and immediately think: what is the smartest way to solve this?

Why Does It Run Every Day for 100 Days?

Problem-solving fluency is a muscle. You cannot build a muscle by going to the gym once a month. You need consistent, daily repetition. That is why DSA is not saved for the last 10 days — it runs every single day from Day 1 to Day 100.

By the end, you will have completed 100 dedicated hours of structured problem-solving practice. This is the competitive readiness required by companies like Google, Amazon, and Microsoft.

What You Actually Learn

Time & Space Complexity — The Language of Efficiency

Before you can judge whether your solution is good, you need a way to measure it. Time complexity tells you how fast your code runs. Space complexity tells you how much memory it uses.

Think of it like this: if you have two ways to drive from Mumbai to Pune, one takes 3 hours and uses half a tank of fuel, the other takes 5 hours and uses a full tank. Time complexity is the hours. Space complexity is the fuel.

Recursion & Problem Decomposition — Divide and Conquer

Recursion is the idea of solving a big problem by breaking it into smaller, identical problems. Imagine you need to count all the files in a folder that has subfolders, which have more subfolders. You can handle each folder the same way — count the files inside, then repeat for each subfolder. That repetitive self-referencing is recursion.

Core Data Structures — The Building Blocks

Data structures are ways of organizing information. Each structure has strengths and weaknesses, and choosing the right one is often the difference between a fast program and a slow one.

- Arrays & Strings: The simplest containers — like a row of boxes in a line
- Hashing: Like an index in a book — instant lookup instead of reading page by page
- Linked Lists: Like a treasure hunt — each clue points to the next one
- Stacks & Queues: Like a stack of plates (last in, first out) vs. a line at a ticket counter (first in, first out)
- Trees: Like a family tree — one root, branching out into children and grandchildren
- Heaps: Like a priority queue — always gives you the most important item first
- Graphs: Like a map of cities and roads — useful for navigation, social networks, and recommendations

Problem-Solving Patterns — The Strategic Toolkit

Just like a chess player has standard openings, a great coder has standard patterns for standard problems.

- Two Pointers & Sliding Window: Used for problems involving sequences — like finding the longest streak of warm days in a weather dataset
- Binary Search: Like looking for a word in a dictionary by opening in the middle, then deciding which half to look in
- Greedy Algorithms: Making the locally best choice at each step — like always picking up the highest-value coin you can see
- Backtracking: Trying all possibilities and backing up when you hit a dead end — like solving a maze
- Dynamic Programming: Remembering solutions to smaller problems so you don't solve them again — like caching directions you've already calculated
- Bit Manipulation: Working at the deepest level of computer memory — used in optimization and cryptography



Every DSA concept you learn directly supports your ability to write AI code. When you implement a neural network, you are working with arrays and matrices. When you design a search system, you use graphs and hashing. When you optimize a model, you need to understand time and space tradeoffs. DSA is not separate from AI — it is underneath it.



Phase 1 — Framework of Machine Learning

Understanding how computers learn from experience

What Is Machine Learning?

Imagine teaching a child to recognize a dog. You don't write a rulebook that says: 'four legs, fur, barks, tail'. Instead, you show the child hundreds of photos of dogs. Over time, the child builds an internal understanding of what a dog looks like.

Machine Learning works the same way. Instead of programming a computer with explicit rules, you feed it thousands of examples. The computer finds patterns on its own and learns to make predictions.



Traditional Programming: You write the rules → computer follows them. Machine Learning: You give examples → computer discovers the rules itself.

The Four Types of Learning

Supervised Learning — Learning with a Teacher

Every example comes with the right answer. You show the system 10,000 photos of emails, labeled either 'spam' or 'not spam'. The model learns to tell them apart. This is how most practical AI systems work today.

Unsupervised Learning — Learning Through Discovery

No labels, no right answers. You give the system raw data and say: 'find patterns'. It might discover that customers naturally cluster into three groups based on their buying habits — even though nobody told it to look for groups. This is how recommendation systems and market segmentation work.

Self-Supervised Learning — Learning from Itself

The model creates its own teaching signal from the data. For example: hide a word in a sentence and ask the model to predict what the missing word is. The text itself becomes the teacher. This is the foundation of how ChatGPT and other large language models are trained.

Reinforcement Learning — Learning from Trial and Error

Like training a dog with treats. The agent takes actions, receives rewards when it does something right and penalties when it doesn't, and gradually learns the best strategy. This is how AlphaGo learned to play Go better than any human, and how robots learn to walk.

Loss Functions — How a Model Knows It Is Wrong

When a model makes a prediction, it needs to know how wrong it was. The 'loss function' is the measuring stick. Think of it like a score in a test — the higher the loss, the worse the model performed.

The model's entire training process is just one goal: reduce the loss score as much as possible. Different types of problems need different measuring sticks:

- **MSE & MAE:** Used when predicting numbers (like house prices). Measures how far off your guess was from the real answer.
- **Cross-Entropy Loss:** Used when choosing between categories (like 'cat or dog'). Measures how confident and correct your probability estimate was.
- **Hinge Loss:** Used by SVMs (a special type of model). Measures how far away the correct answer is from the decision boundary.
- **KL Divergence:** Measures how different two probability distributions are — used when comparing what a model thinks the world looks like vs. what it actually looks like.

Important ML Algorithms

These are the workhorses of traditional machine learning. Each one was designed for a specific type of problem:

- **Linear & Logistic Regression:** The simplest possible models. Linear regression draws a straight line to predict a number. Logistic regression draws a boundary to classify things into categories. These are the first tools you reach for — if they work, great. If not, you upgrade.
- **Decision Trees & Ensembles:** A tree of yes/no questions that leads to a decision. 'Is the person over 40? Yes → Is their cholesterol high? Yes → High heart risk.' Ensembles combine hundreds of trees for much better accuracy.
- **Support Vector Machines:** Finds the widest possible gap between two categories. Like drawing a road as far from both sidewalks as possible.
- **K-Nearest Neighbours:** Classifies something based on its neighbours. If 7 of the 10 most similar examples are cats, this new one is probably a cat too.
- **Naive Bayes:** Uses probability theory to classify things. Fast, simple, surprisingly effective for text classification like spam filtering.

Metric Analysis — How Do We Know the Model Is Actually Good?

A model that says 'not spam' for every single email would be 99% accurate — if only 1% of emails are spam. But that model is useless. This is why accuracy alone is not enough.

- **Precision:** Of all the times the model said 'this is spam', how often was it right?
- **Recall:** Of all the actual spam emails, how many did the model catch?
- **F1 Score:** A balance between precision and recall — use this when both matter equally.
- **ROC-AUC:** A measure of how well the model separates two classes across all possible thresholds.
- **RMSE & R²:** For regression tasks — how close are your number predictions to the real values?

- Confusion Matrix: A visual breakdown of every type of correct and incorrect prediction.
- Cross-Validation: Tests the model on multiple different slices of data to make sure it is not just memorizing the training examples.



Phase 1 is the foundation of everything that follows. Deep Learning (Phase 2) is just a more powerful and complex version of Machine Learning. Before you can understand WHY deep learning is needed, you must understand WHERE machine learning falls short. Every concept here — loss functions, evaluation metrics, learning types — will appear again and again throughout the program.

MARGAM ACADEMY



Phase 2 — Framework of Deep Learning

When machine learning gets a brain upgrade

What Is Deep Learning and Why Did It Emerge?

Traditional machine learning works well for structured data — spreadsheets, numbers, simple categories. But what about photos? Audio? Language? These types of data are far too complex for simple algorithms to understand.

Deep Learning solves this by using networks of artificial 'neurons' — loosely inspired by the human brain — that can learn to recognize incredibly complex patterns. The 'deep' in deep learning refers to the many layers of these neurons stacked on top of each other.



Analogy: Traditional ML is like looking at a painting and describing it by color percentages. Deep Learning is like actually seeing the painting — the shapes, the shadows, the emotion, the meaning.

The Forward Pass — How Information Flows Through a Network

Imagine a factory assembly line. Raw material enters at one end. At each station, workers transform it a little. At the end of the line, a finished product emerges.

A neural network works the same way. Raw input (like an image) enters the first layer. Each layer applies a mathematical transformation. By the final layer, the raw pixels have become a confident answer: 'This is a cat.'

This journey from input to output is called the Forward Pass. Understanding it in detail is essential because it forms the basis of everything else — without knowing how predictions are made, you cannot understand how the model learns to improve them.

Activation Functions — Giving the Network Its Power

Here is a problem: if every layer just applies a linear (straight-line) transformation, then stacking 100 layers is mathematically identical to having just 1 layer. You gain nothing by going 'deep'.

Activation functions fix this by introducing non-linearity — bending and shaping the data in ways a straight line cannot. This is what allows deep networks to learn complex, curved patterns.

- Sigmoid & Tanh: Older functions that squish values into a small range. Useful for probabilities, but they cause a problem called 'vanishing gradient' in deep networks.
- ReLU (Rectified Linear Unit): Simply turns all negative values to zero. Sounds too simple — but it is remarkably effective and fast to compute. The default choice for most hidden layers.
- LeakyReLU & ELU: Small improvements to ReLU that allow a tiny signal even for negative values.

- GELU & SiLU: Smoother, more sophisticated versions used in modern Transformers like GPT and BERT.



The vanishing gradient problem: in very deep networks, the error signal gets smaller and smaller as it travels backwards through layers — like a whisper that fades to nothing. Modern activation functions and normalization techniques were invented specifically to solve this problem.

Loss Functions in Deep Learning — More Complex Objectives

Phase 1 introduced basic loss functions for regression and classification. Deep learning extends these to much more complex tasks:

- Focal Loss: Standard cross-entropy treats all mistakes equally. But what if 99% of your images are 'background' and 1% are 'objects'? Focal loss puts more attention on the rare, hard examples.
- Contrastive & Triplet Loss: Used when you want the model to learn similarity. 'These two faces are the same person. This face is different.' Used in face recognition, product similarity, and search.
- CTC Loss (Connectionist Temporal Classification): Used for speech-to-text and handwriting recognition, where the input and output are different lengths.
- Reconstruction Loss: Used in Autoencoders and generative models. The model tries to recreate its input exactly — the difference between original and reconstruction is the loss.

Overview of Deep Learning Architectures

Different types of data need different types of networks. This is one of the most important insights in deep learning:

- MLPs (Multi-Layer Perceptrons): The basic form — fully connected layers. Good for tabular data. Used as building blocks inside other architectures.
- CNNs (Convolutional Neural Networks): Designed for images. Instead of looking at the whole image at once, they scan it with small filters — detecting edges, shapes, textures, then combining those into higher-level features. Used in photo apps, medical imaging, self-driving cars.
- RNNs & LSTMs (Recurrent Neural Networks): Designed for sequences — text, audio, time series. They have a 'memory' that carries information from step to step. LSTMs are an improvement that can remember information over longer sequences.
- Autoencoders: Networks that compress data into a small representation and then reconstruct it. Used for anomaly detection, noise removal, and as building blocks for generative models.
- GANs (Generative Adversarial Networks): Two networks compete — one generates fake data, the other tries to detect fakes. Through this competition, the generator gets incredibly good at making realistic images, videos, or audio.
- Transformers: The architecture that changed everything. [Preview here](#), [deep dive in Phase 3](#).

Optimizations — Making Training Actually Work

Having a network architecture is not enough. Training it — actually getting it to learn — requires a careful combination of optimization techniques.

- **Gradient Descent (SGD, Adam, AdamW):** The core learning algorithm. After each prediction, calculate how wrong the model was, then nudge every parameter slightly in the direction that would have made it more right. Repeat millions of times. Adam and AdamW are smarter versions that adapt the nudge size automatically.
- **Learning Rate Scheduling:** The learning rate controls how big each nudge is. Too big and you overshoot. Too small and training takes forever. Scheduling gradually reduces the learning rate as training progresses.
- **Batch Normalization & Layer Normalization:** Standardize the values flowing through the network at each layer. This prevents extreme values that destabilize training and dramatically speeds up convergence.
- **Dropout:** During training, randomly turn off some neurons. This forces the network to not over-rely on any single neuron — making it more robust and general.
- **Weight Initialization:** How you set the initial parameter values matters enormously. Bad initialization can cause training to fail before it even starts.
- **Gradient Clipping:** If gradients explode to very large values, they can destabilize training. Clipping caps them at a safe maximum.



Deep Learning is the engine inside modern AI. Everything in Phase 3 (Transformers), Phase 5 (GenAI), and Phase 6 (System Design) is built on top of the foundations laid here. The forward pass, backpropagation, activation functions, and optimizers you learn in Phase 2 are operating inside every ChatGPT, Gemini, and Claude model in the world right now.



Phase 3 — Overview of Transformers

The architecture that powers ChatGPT, Claude, Gemini, and every modern AI

What Are Transformers and Why Do They Matter?

Before 2017, the best AI systems for language used RNNs — networks that processed words one at a time, in order, like reading a sentence left to right. They were slow, struggled with long texts, and couldn't be parallelized.

Then came the Transformer — an architecture based on a revolutionary idea called 'attention'. Instead of reading left to right, a Transformer looks at all words simultaneously and figures out which words are most relevant to each other.

This single architectural change unlocked GPT, BERT, T5, Claude, Gemini, LLaMA, and virtually every major AI system of the last five years.



Old RNN: reads 'The bank can guarantee deposits will eventually cover future tuition costs' word by word — and by the time it gets to 'costs', it has nearly forgotten 'bank'.
Transformer: reads the entire sentence at once and immediately connects 'bank' with 'deposits', 'guarantee', and 'cover'.

Building Block 1: Tokenization — Breaking Language Into Pieces

Computers cannot read words. They only understand numbers. Tokenization is the process of converting text into numerical tokens (pieces) that a model can process.

'Hello, world!' might become [15496, 11, 995, 0] — each number representing a word, sub-word, or character. The vocabulary (the list of all possible tokens) is typically 50,000 to 100,000 items.

Why sub-words? Because if you only tokenize whole words, the vocabulary becomes impossibly large and rare words are never learned. Sub-word tokenization (BPE, WordPiece, SentencePiece) allows the model to handle any word by combining familiar sub-pieces.

Building Block 2: Self-Attention — The Revolutionary Idea

Attention is the ability to look at all words at once and decide which ones are relevant to each other. Here is how it works conceptually:

For each word (called a Query), the model looks at every other word (as Keys) and calculates a relevance score. Words with high scores get more 'attention'. Then it uses those attention scores to create a weighted summary of all words (Values).

Example: In the sentence 'The trophy did not fit in the suitcase because it was too big', the word 'it' attends strongly to 'trophy' (not 'suitcase') because the model learns that 'too big' fits with 'trophy'.



The quadratic complexity tradeoff: Self-attention must compare every word to every other word. For a sentence of 100 words, that is $100 \times 100 = 10,000$ comparisons. For 1,000 words, 1,000,000 comparisons. This is why processing very long documents is expensive — and why researchers are constantly working on making attention more efficient.

Building Block 3: Multi-Headed Attention — Multiple Perspectives at Once

One attention operation can only focus on one type of relationship at a time. But language has many relationship types simultaneously — syntactic (grammar), semantic (meaning), coreference (which 'it' refers to what), and more.

Multi-headed attention runs multiple attention operations in parallel, each learning to focus on different relationship types. The results are then combined, giving the model a rich, multi-dimensional understanding of context.

Think of it like a panel of experts each reading the same document, each looking for different things, and then pooling their insights.

Building Block 4: Normalization — Keeping Things Stable

As data flows through dozens of layers, values can become extremely large or extremely small. Normalization layers periodically re-center and re-scale values, keeping training stable.

Transformers typically use Layer Normalization rather than Batch Normalization, because it works well for variable-length sequences and doesn't require large batch sizes.

Building Block 5: The Loss Function in LLMs — Next-Token Prediction

How is a language model trained? The objective is beautifully simple: given the words so far, predict the next word. The model receives 'The sky is' and must predict 'blue' (or 'clear', or 'dark' — with appropriate probabilities for each).

This single objective, repeated across trillions of examples from the internet, causes the model to develop an astonishing range of capabilities — grammar, facts, reasoning, coding, translation — all emerging from next-word prediction.

Building Block 6: Sampling Strategies — How the Model Generates Text

After training, when generating text, the model outputs a probability distribution over all possible next tokens. How do you choose which one to pick?

- Greedy Decoding: Always pick the highest-probability token. Fast but repetitive — leads to generic, boring outputs.

- Beam Search: Keep track of the top-k best sequences at each step. Better quality but slower.
- Top-K Sampling: Randomly sample from the top K most likely tokens. Adds variety.
- Top-P (Nucleus) Sampling: Sample from the smallest set of tokens whose combined probability exceeds P. More adaptive than top-K.
- Temperature: Controls how 'spiky' the distribution is. High temperature = more random and creative. Low temperature = more focused and deterministic.



Phase 3 is the pivot point of the entire program. Everything in Phase 1 and Phase 2 was building toward this — and everything in Phase 5 (GenAI Engineering) builds directly on top of it. When you use ChatGPT, you are using a Transformer. When you build a RAG system or an AI agent, you are using a Transformer. This phase is arguably the most important in the entire curriculum.



Phase 4 — Backend & AI Deployment

Building the bridge from a trained model to a live product

Why Can't a Model Just Live in a Notebook?

When data scientists train a model, it typically exists inside a Jupyter Notebook — a file on their personal computer. Nobody else can use it. It cannot receive requests. It cannot serve millions of users. It is useful only to the person sitting in front of it.

To turn that model into a real product, you need to build an API — a web service that accepts inputs (like a question) and returns outputs (like an answer). This is what Phase 4 teaches.



Think of an API like a restaurant's ordering system. The kitchen (the model) does the actual cooking. But without a waiter and a menu (the API), nobody can order anything.

FastAPI — The Industry Standard for AI Services

FastAPI is a Python framework for building web services. It has become the industry standard for AI backends because it is fast, modern, and easy to use.

REST API Design — The Language of the Web

REST is a set of conventions for how web services communicate. You send a request with a URL and some data. The server processes it and sends back a response. Every web application you use — social media, email, banking — is built on REST APIs underneath.

Model Serving — Connecting Your Model to the World

Model serving is the process of loading your trained AI model and wrapping it in a web service that can receive inputs, run them through the model, and return predictions. This is the critical step that converts a research artifact into a usable product.

Authentication & Authorization — Protecting Your Service

Not everyone should be able to use your AI service for free. Authentication verifies who you are (like logging in). Authorization determines what you are allowed to do (like how many requests per day). These are essential for commercial AI products.

Production API Structure — Built to Scale

A personal project can be messy. A production API serving millions of users cannot. This section teaches the structure and patterns that make APIs reliable, maintainable, and scalable.



Every AI project you build in Phase 5 will need a backend. Every RAG system, every agent, every GenAI application — all of them need to be accessible through an API. Phase 4 gives you the infrastructure skills to deploy everything you learn next.

MARGAM ACADEMY



Phase 5 — GenAI Engineering

The crown jewel — building, using, and operating real AI products

What Is Generative AI?

Generative AI refers to AI systems that can create new content — text, images, code, audio, video — rather than just classifying or predicting existing content.

ChatGPT writes essays. DALL-E creates images from text descriptions. GitHub Copilot generates code. These are all generative AI systems, and they are powered by the transformer architecture you learned in Phase 3.

Phase 5 is the largest phase of the program because it covers the full stack of what it means to be a GenAI engineer — from understanding how LLMs are built, to using them through APIs, to orchestrating them in complex applications, to deploying and monitoring them in production.

Part A: The Builder Perspective — How Are LLMs Actually Made?

Understanding how a large language model is built is what separates an engineer from a user. Even if you never train a model from scratch, knowing the process helps you evaluate models, compress them, and understand their capabilities and failures.

- **Transformer Internals:** A deeper look at the architecture from Phase 3 — understanding exactly what happens at each layer, how parameters are organized, and how scale affects quality.
- **Pretraining:** The process of training on trillions of tokens of text from the internet. This is where the model acquires its broad knowledge of language, facts, and reasoning.
- **Optimization at Scale:** Training large models requires specialized hardware (GPU clusters), distributed computing, and careful engineering to stay stable across billions of parameter updates.
- **Model Compression:** Large models are expensive to run. Techniques like quantization (reducing numerical precision), pruning (removing unnecessary connections), and distillation (training a small model to mimic a large one) make models smaller and faster without sacrificing too much quality.
- **LLM Evaluation:** How do you know if one model is better than another? This covers benchmarks, human evaluation, automated metrics, and the tricky business of measuring qualities like reasoning, factuality, and safety.
- **Inference Optimization:** After training, how do you serve the model as cheaply and quickly as possible? Techniques like batching, caching, and speculative decoding can dramatically reduce cost.

Part B: The Practitioner Perspective — Using LLMs in Real Applications

This is the day-to-day work of a modern AI engineer. Most of the time, you are not training new models — you are using existing ones to build products.

LLM APIs — Accessing AI as a Service

OpenAI, Anthropic, and Google offer their models through APIs. You send a request (your prompt + context), and they send back a generated response. Understanding how to use these APIs effectively — with proper error handling, token management, and cost awareness — is foundational.

Open-Source LLMs — Building Without Dependency

Models like LLaMA (Meta) and Mistral can be downloaded and run on your own hardware. This gives you privacy, cost control, and the ability to fine-tune for specific domains. Trade-off: you are responsible for hosting and maintaining them.

LangChain — Orchestrating AI Workflows

LangChain is a framework for building complex AI applications. Instead of writing raw API calls, you chain together components: 'retrieve relevant documents' → 'format them into a prompt' → 'call the LLM' → 'parse the response' → 'take an action'. Like LEGO blocks for AI applications.

Prompt Engineering — Communicating with AI

The way you phrase your request to an LLM dramatically affects the quality of the response. Prompt engineering is the art and science of crafting inputs that elicit the best outputs. Techniques include chain-of-thought prompting (asking the model to show its reasoning), few-shot examples (showing the model what you want), and system prompts (setting the model's role and context).

DSPy — Automating Prompt Optimization

Instead of manually crafting prompts, DSPy uses machine learning to optimize them automatically. You define what you want (the task and metric), and DSPy finds the best prompts and configurations to achieve it.

AI Agents — AI That Takes Actions

An AI agent is not just a question-answering system — it is a system that can take actions in the world. It can search the web, write and execute code, send emails, query databases, book appointments. The LLM acts as the 'brain' that decides what action to take next based on its goal and the current state.

Think of the difference between asking a question and hiring an assistant. An assistant doesn't just answer — they act.

Vector Databases — AI Memory

LLMs have a context window — they can only 'remember' a limited amount of text at once. Vector databases solve the memory problem by storing information as mathematical vectors (embeddings) and enabling fast similarity search.

When you ask a question, the system retrieves the most relevant stored information and includes it in the prompt. This allows AI systems to access vast knowledge bases without needing to fit everything into the context window.

Multimodal Usage — AI That Sees and Hears

Modern LLMs can process not just text, but images, audio, documents, and more. A multimodal AI can look at a photo and describe what it sees, read a chart and explain the trend, or listen to a recording and transcribe it.

Part C: RAG — Retrieval-Augmented Generation

RAG is the dominant architecture for building AI applications that need to answer questions based on specific, up-to-date, or proprietary information.

The problem RAG solves: LLMs are trained on data up to a certain date. They don't know about events after that date. They don't know about your company's internal documents. They can hallucinate (make up plausible-sounding but false information).

The RAG solution: Before answering a question, retrieve relevant documents from a database and include them in the prompt. Now the model answers based on real, specific information rather than just its training data.



RAG in everyday terms: Imagine asking a consultant a question. Without RAG, they answer from memory alone. With RAG, they first look up the relevant files, read them, and then answer based on what they actually found.

- Traditional RAG: Retrieve text → Include in prompt → Generate answer. The foundation.
- Multi-Modal RAG: Retrieve not just text but also images, tables, charts, and PDFs.
- Knowledge Graph RAG: Use structured knowledge graphs (networks of facts and relationships) as the retrieval source — better for complex, multi-hop reasoning.
- Chunking Strategies: Documents must be split into pieces before being stored. How you chunk matters enormously — too small and you lose context, too large and retrieval becomes imprecise.
- RAG Evaluation: Measuring whether the retrieved information was relevant, and whether the generated answer was faithful to the source and actually correct.

Part D: MCP — Model Context Protocol

As AI agents become more powerful, they need a standardized way to interact with external tools and systems — databases, APIs, file systems, browsers. MCP is an emerging standard that defines how AI agents communicate with these external resources.

Think of MCP as a universal remote control for AI agents. Instead of each agent needing custom code for every tool it might use, MCP provides a standard interface that any tool can implement.

Part E: LLMOps — Operating AI Systems in Production

Deploying an AI system is not a one-time event. Models drift, degrade, generate unexpected outputs, and need constant monitoring.

- **Evaluation Frameworks:** Automated systems that continuously test model outputs against quality benchmarks.
- **Monitoring:** Real-time tracking of response quality, latency, cost, and error rates.
- **Model Versioning:** When you update a model, you need to track which version is running in production and be able to roll back quickly if something goes wrong.
- **Logging & Observability:** Recording what inputs come in and what outputs go out — essential for debugging and improvement.
- **Performance Tracking:** Measuring whether the system is actually helping users achieve their goals over time.

Part F: MLOps — The Engineering Discipline Behind Reliable ML

MLOps is the practice of applying software engineering discipline to machine learning systems. Without it, models exist only as experiments that cannot be reliably reproduced, scaled, or maintained.

- **Linux Basics:** The foundation of all cloud and server infrastructure. Every production AI system runs on Linux servers.
- **Docker:** Package your model and all its dependencies into a 'container' that runs identically on any machine. No more 'it works on my laptop'.
- **Kubernetes:** Orchestrate hundreds of Docker containers across multiple servers. Automatically scales up when traffic increases and down when it decreases.
- **MLflow:** Track all your experiments — which model, which data, which hyperparameters, which results. Essential for reproducibility.
- **Kubeflow:** Kubernetes-native pipelines for ML workflows — from data ingestion to training to deployment.
- **CI/CD:** Continuous Integration / Continuous Deployment. Automatically test and deploy new model versions when changes are made.
- **Prometheus & Grafana:** Industry-standard tools for monitoring and visualizing system performance in real time.



Phase 5 is where everything comes together. The math from Phase 1, the deep learning from Phase 2, the transformer architecture from Phase 3, and the API skills from Phase 4 — all of them are assembled here into real, working, deployed AI products. This phase is the reason the entire program exists.



Phase 6 — System Design

Building AI that works at scale — for millions of users

What Is System Design and Why Does It Matter?

Imagine you build a fantastic AI chatbot. 10 people try it — it works perfectly. Then you appear on a tech show, and 100,000 people try to use it at the same time. Your server crashes. The system fails. Your product is destroyed in its biggest moment.

System design is the discipline of building systems that don't fail under load — that can handle millions of users, recover from failures, scale up on demand, and maintain quality over time.

This is the capstone skill of the program. It requires everything that came before it. You cannot design a good AI system without understanding ML, deep learning, transformers, APIs, and MLOps.

Core Primitives — The Foundation of All Distributed Systems

Load Balancing — Spreading the Work

When 100,000 users send requests simultaneously, a single server cannot handle them all. A load balancer distributes incoming requests across multiple servers, ensuring no single machine becomes a bottleneck. Like having multiple checkout lanes at a supermarket.

Caching — Answering Without Thinking

Many AI queries are repetitive. 'What is the capital of France?' does not need a fresh model inference every time. Caching stores recent results and serves them instantly if the same query appears again. Dramatically reduces cost and latency.

Distributed Systems Basics — When One Machine Is Not Enough

Modern AI systems run across tens, hundreds, or thousands of machines working together. Understanding the principles of distributed systems — consistency, availability, fault tolerance, partitioning — is essential for designing systems that remain reliable even when individual components fail.

Scalability Patterns — Growing Without Breaking

Horizontal scaling (adding more servers), vertical scaling (making servers more powerful), sharding (splitting data across machines), and replication (copying data for redundancy) are the core patterns for building systems that grow with demand.

ML System Design — Production Training Pipelines

There is a big difference between training a model in a notebook and running a production training pipeline that retrains the model every week on new data, tracks every experiment, and deploys new versions automatically.

- **Training Pipeline Design:** Automating the entire process from raw data ingestion to trained model artifact — with validation at each step.
- **Feature Store:** A centralized repository of processed features (inputs) that can be reused across multiple models, ensuring consistency between training and serving.
- **Drift Monitoring:** Over time, the real-world data distribution changes. Yesterday's model trained on last year's data may perform poorly on today's inputs. Drift monitoring detects this degradation early.
- **Experiment Tracking:** Systematic recording of every training run — hyperparameters, dataset version, metrics — so results are reproducible and comparable.

GenAI System Design — Unique Challenges of LLM Systems

LLM-based systems have unique architectural requirements that differ from traditional ML systems:

- **RAG Architecture at Scale:** Handling millions of documents, sub-second retrieval, and maintaining freshness as documents are updated — at production scale.
- **Agent System Design:** Multi-agent systems where specialized agents collaborate, pass tasks to each other, maintain shared state, and recover from individual failures.
- **LLM Serving Architecture:** Efficiently batching requests, managing GPU memory, handling variable-length outputs, and distributing model weights across multiple GPUs.
- **Scaling LLM Inference:** The economics of serving LLMs are very different from traditional software. GPU time is expensive. This section covers the architectural choices — model quantization, speculative decoding, KV cache management — that determine whether a product is economically viable.



System Design is the final layer of the program. Everything you have built across 90+ days comes together here — not as isolated components, but as a unified, production-grade architecture. This is the skill that commands the highest salaries and the most senior engineering titles.



Connecting All the Dots

How every phase leads to the next — the full learning arc

The Learning Arc: A Single Continuous Journey

This program is not a collection of separate topics. Every phase is a logical step forward from the previous one. Here is the complete chain:

Phase 1: ML Foundations → Understand **WHAT** learning is and **HOW** models are measured

Phase 2: Deep Learning → Understand **WHY** we need neural networks and **HOW** they learn

Phase 3: Transformers → Understand the **ARCHITECTURE** that powers all modern AI

Phase 4: FastAPI → Understand **HOW** to expose models as real web services

Phase 5: GenAI Engineering → Understand **HOW** to **BUILD** with LLMs in production

Phase 6: System Design → Understand **HOW** to **SCALE** everything to millions of users



DSA runs underneath all of this — every single day. The problem-solving muscle you build through daily DSA practice is the same muscle you use to write efficient data pipelines, optimize inference systems, and crack the coding interviews that get you through the door.

Why the Order Matters

You might wonder: why not just jump to GenAI? Why spend weeks on classical ML and deep learning before touching ChatGPT?

Here is the honest answer: you can use ChatGPT without knowing any of this. But you cannot engineer, evaluate, debug, or improve AI systems without the foundations. When your RAG system hallucinates, you need to know why — is it the retrieval? The chunking? The prompt? The model temperature? Without the foundations, you can only guess. With them, you can diagnose precisely and fix systematically.

The program is designed so that by the time you reach GenAI, you understand every component that makes it work. You are not a user who presses buttons — you are an engineer who understands the machine.



Career Outcomes

The 25 roles you can target after 100 days

What the Job Market Looks Like

The AI job market in 2024-2025 is one of the most dynamic in tech history. Companies are building AI teams from scratch, upgrading existing products with AI capabilities, and searching aggressively for engineers who can bridge the gap between AI research and production engineering.

The graduates of this program are equipped for roles across four distinct career tracks:

Track 1: Core AI & ML Engineering

These are the foundational AI roles — the engineers who build and maintain ML systems at companies across every industry. The depth of Phases 1-3 makes graduates directly competitive for these positions.

- Machine Learning Engineer — Builds and maintains ML pipelines, trains models, deploys them to production
- Deep Learning Engineer — Specializes in neural network architectures for vision, audio, or language
- AI Engineer — Broad role covering the full stack from data to deployment
- Applied Scientist / Applied ML Engineer — Research-leaning role that applies novel ML techniques to real business problems
- Computer Vision Engineer — Builds systems that understand images and video
- NLP Engineer — Builds systems that understand and generate language
- Research Engineer — Bridges academic research and production implementation

Track 2: GenAI & LLM Specialization

These are the fastest-growing roles in tech right now. Phase 5 of the program is specifically designed to make graduates competitive for the entire GenAI engineering track.

- Generative AI Engineer — Builds products powered by generative models
- LLM Engineer — Specializes in fine-tuning, evaluating, and deploying large language models
- Prompt Engineer (Advanced) — Designs complex prompt systems, evaluation frameworks, and optimization pipelines
- RAG Engineer — Builds retrieval-augmented generation systems for knowledge-grounded applications
- AI Agent Developer — Designs and builds autonomous AI systems that take real-world actions
- Conversational AI Engineer — Builds chatbots, voice assistants, and dialogue systems
- Multimodal AI Engineer — Builds systems that combine text, vision, audio, and other modalities

Track 3: MLOps & Infrastructure

As AI systems become business-critical, the need for engineers who can operate them reliably has exploded. The MLOps and LLMOps content in Phase 5 directly targets these roles.

- MLOps Engineer — Builds and maintains the infrastructure that makes ML training and deployment reliable
- LLMOps Engineer — Specializes in operating large language model systems in production
- AI Platform Engineer — Builds internal platforms that enable data scientists and ML engineers to work faster
- ML Infrastructure Engineer — Designs the compute infrastructure underlying AI systems
- AI DevOps Engineer — Applies DevOps principles to AI development workflows
- Model Deployment Engineer — Specializes in the last mile — getting models from notebooks to production

Track 4: System Design & Architecture

Senior engineering roles that require the full breadth of the program — especially Phase 6. These are the roles with the highest compensation and the most strategic impact.

- AI Systems Architect — Designs high-level architectures for complex AI systems
- ML Systems Designer — Focuses on the design of scalable, reliable ML platforms
- Backend Engineer (AI-Focused) — Builds the server-side infrastructure for AI applications
- AI Product Engineer — Bridges technical capability and product experience
- Data & AI Platform Architect — Designs the data and AI infrastructure for entire organizations



Summary: What You Will Know After 100 Days

A plain-English recap of every capability built

Capability	What It Means in Plain Language
Understand ML & DL	You know how machines learn from data, why neural networks are powerful, and how to measure whether a model is actually good.
Understand Transformers	You know how ChatGPT, Claude, and Gemini work at an architectural level — not just that they 'use AI'.
Build with LLM APIs	You can integrate OpenAI, Anthropic, or Gemini into any application and handle edge cases professionally.
Build RAG Systems	You can build AI applications that answer questions based on real documents — without hallucinating.
Build AI Agents	You can build autonomous AI systems that browse the web, write code, query databases, and take real actions.
Deploy AI to Production	You can take any model and make it available as a web service that real users can access.
Monitor & Maintain AI	You know how to detect when a model degrades, how to log its behaviour, and how to version and update it safely.
Design Scalable Systems	You can design AI systems that work for millions of users, not just for a demo.
Crack Coding Interviews	100 hours of DSA practice makes you competitive for the technical interview process at top-tier companies.
Read Research Papers	You have developed the vocabulary and conceptual framework to understand and apply findings from the latest AI research.



100 Days. 1 Hour DSA. 1 Hour AI. Every Day. Real Projects. Real Career. By the end, you are not just someone who has heard of AI — you are an engineer who builds it.